

Aula 7

- Barramentos paralelo *vs* barramentos série
- Barramentos série
 - Princípio de funcionamento
 - Sincronização de relógio entre transmissor e recetor
 - Modos de transmissão de dados: transmissão orientada ao bit, transmissão orientada ao byte
 - Topologias de ligação
 - Elementos de uma ligação série

José Luís Azevedo, Bernardo Cunha, Tomás O. Silva, P. Bartolomeu

Introdução

- Barramentos: interligação dos blocos de um sistema de computação
 - CPU, memória, unidades de I/O
- Tipos de dispositivos ligados a um barramento:
 - **Bus Master** – Dispositivo que pode iniciar e controlar uma transferência de dados (exemplos: Processador, Módulo de I/O com DMA)
 - **Bus Slave** – Dispositivo que só responde a pedidos de transferências de dados, i.e., não tem capacidade para iniciar uma transferência (exemplos: Memória, Módulo de I/O sem DMA)
- **Barramento de um só Master:** só há um dispositivo no barramento com capacidade para iniciar e controlar transferências de dados (portão)
- **Barramento Multi-Master:** mais que um dispositivo capaz de iniciar e controlar transferências de dados (exemplos: vários CPUs, 1 ou mais controladores de DMA, um ou mais módulos de I/O com DMA) só pode haver um master de cada vez

Introdução

o que vimos até agora

- **Barramentos paralelo:** os dados são transmitidos em paralelo (através de N Linhas). Incluem:

os dados são transferidos de uma vez só

- **Barramento de dados:**

- Suporta a transferência de informação entre os blocos
- O número de linhas (largura do barramento) determina quantos bits podem ser transferidos simultaneamente; a largura do barramento é um fator determinante no desempenho do sistema

- **Barramento de endereços:**

- Especifica a origem/destino da informação
- O número de linhas define a dimensão do espaço de endereçamento (determina a capacidade máxima de memória que o sistema pode ter: 2^N palavras, sendo N o número de bits do barramento de endereços)

- **Barramento de controlo:**

- Conjunto de sinais que especificam operações, sinalizam eventos, efetuam pedidos, ...

depende da operação



sem controlados pelo master

Introdução

- A transmissão paralela, com relógio comum, a débitos elevados coloca problemas de várias ordens, nomeadamente:
 - Controlo do tempo de "skew" nas linhas do barramento
 - Dificuldade em anular/minimizar a interferência provocada por fontes de ruído externas
 - Interferência mútua, isto é, entre sinais adjacentes ("crosstalk")
 - Elevado número de fios de ligação e custo associado
 - Fichas de ligação volumosas e caras (possivelmente com contactos dourados)

- **Barramentos série:** os dados são serializados no transmissor, ou seja, transmite-se 1 bit de cada vez (tipicamente 1 bit a cada ciclo de relógio)

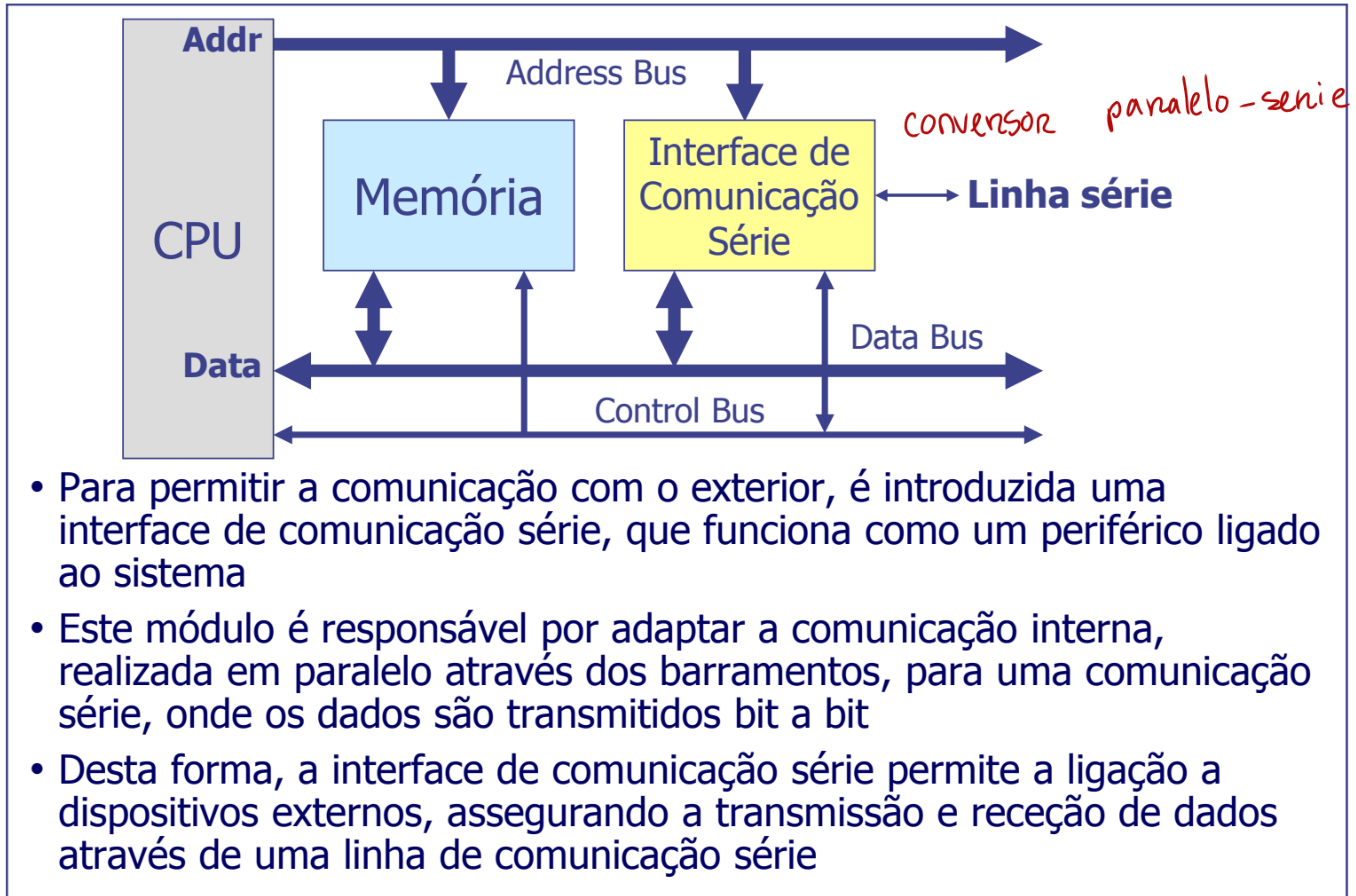
- **Comunicação série**

1 linha = freq. mto mais altas
ent nem fica mais lento por enviar
1 de cada vez

Quando se tem muitas linhas em freq. altas o programa tem atrasos

uma linha condiciona o estado de outra

Comunicação série



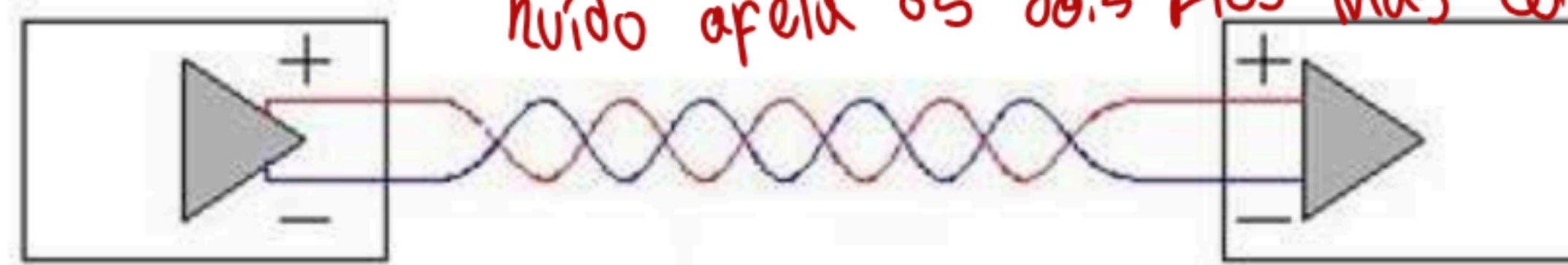
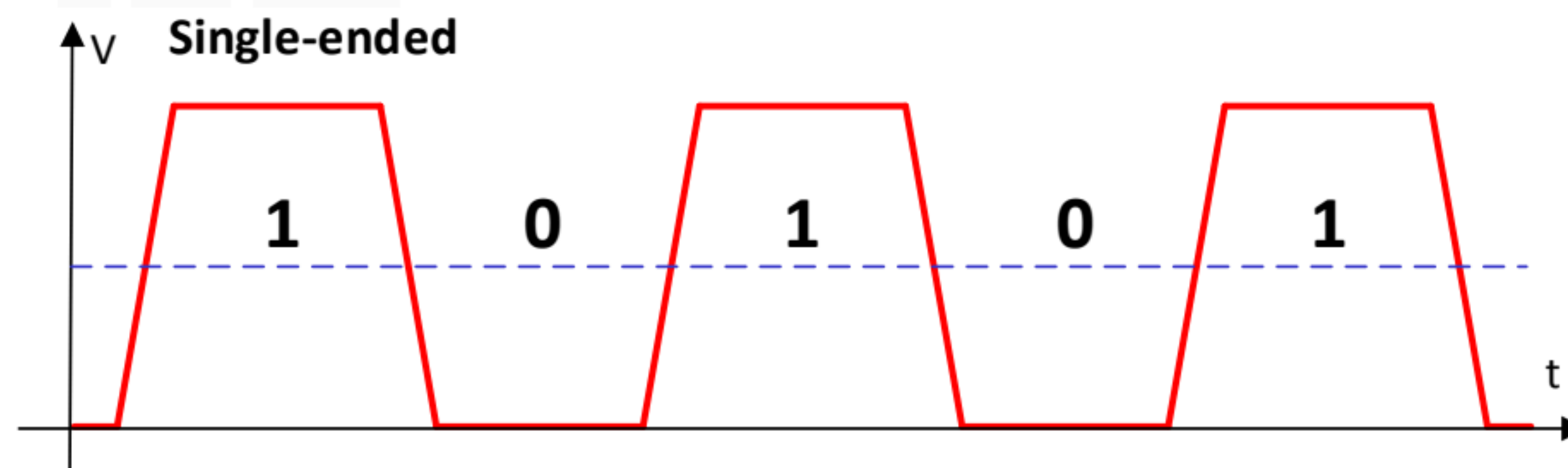
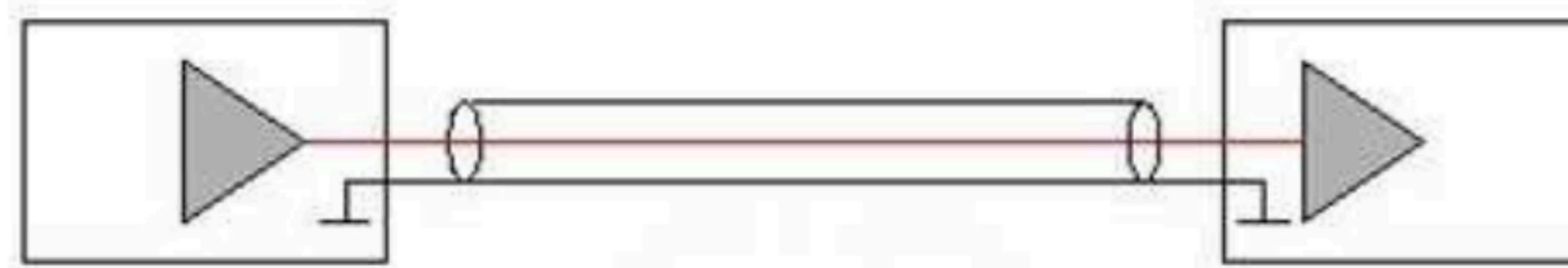
Comunicação série

- Vantagens dos barramentos série (ao nível físico):
 - Ligação de cablagem mais simples
 - Diminuição de custos de interligação
 - Possibilidade de transmissão a distâncias elevadas (especialmente com transmissão diferencial)
 - Débito elevado (frequências de relógio mais elevadas)
- Transmissão "single-ended":
 - Transmissão do sinal digital através de um único fio, sendo o outro fio utilizado como referência de tensão ("ground")
- Transmissão diferencial: *dois fios mas para sinal*
 - Transmissão utilizando dois sinais (geralmente complementares), cada um no seu próprio condutor
 - O recetor interpreta o nível lógico do sinal recebido através da diferença de tensão entre os dois condutores
 - A transmissão diferencial oferece maior imunidade ao ruído e é menos afetada por interferência externa

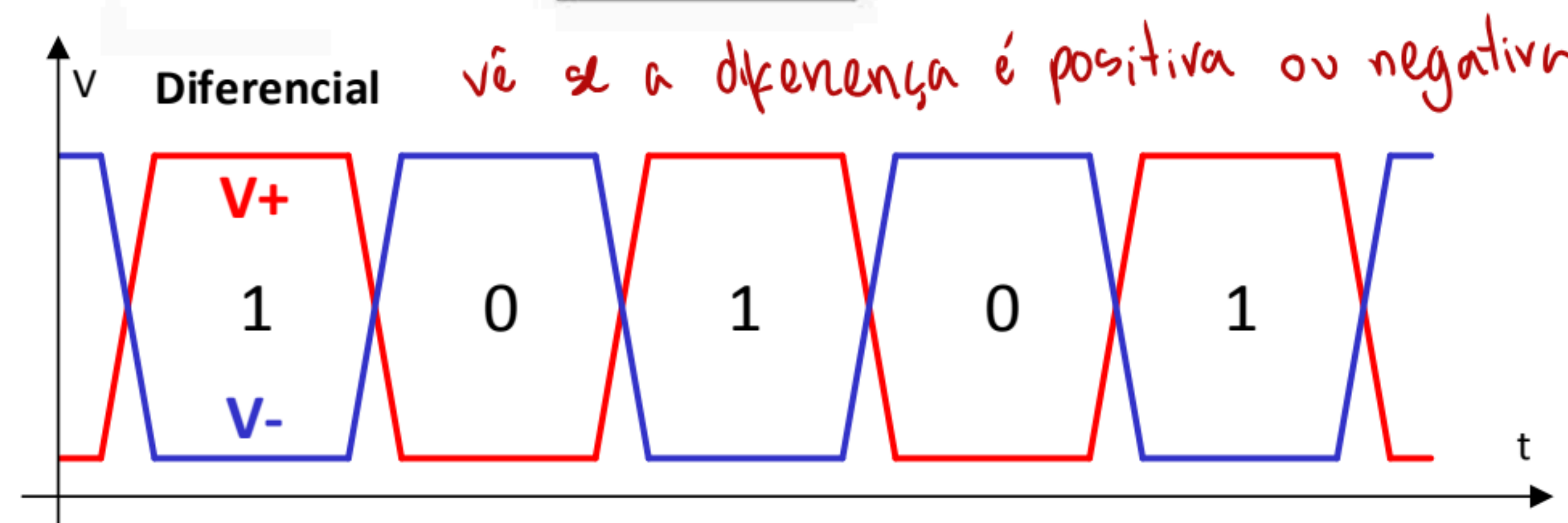
USB usa →

discriminação do nível lógico é feita por diferença entre os dois

Transmissão "single ended" | Transmissão "diferencial"



ruído afeta os dois fios mas como só queremos saber a diferença entre os dois então não afeta nada

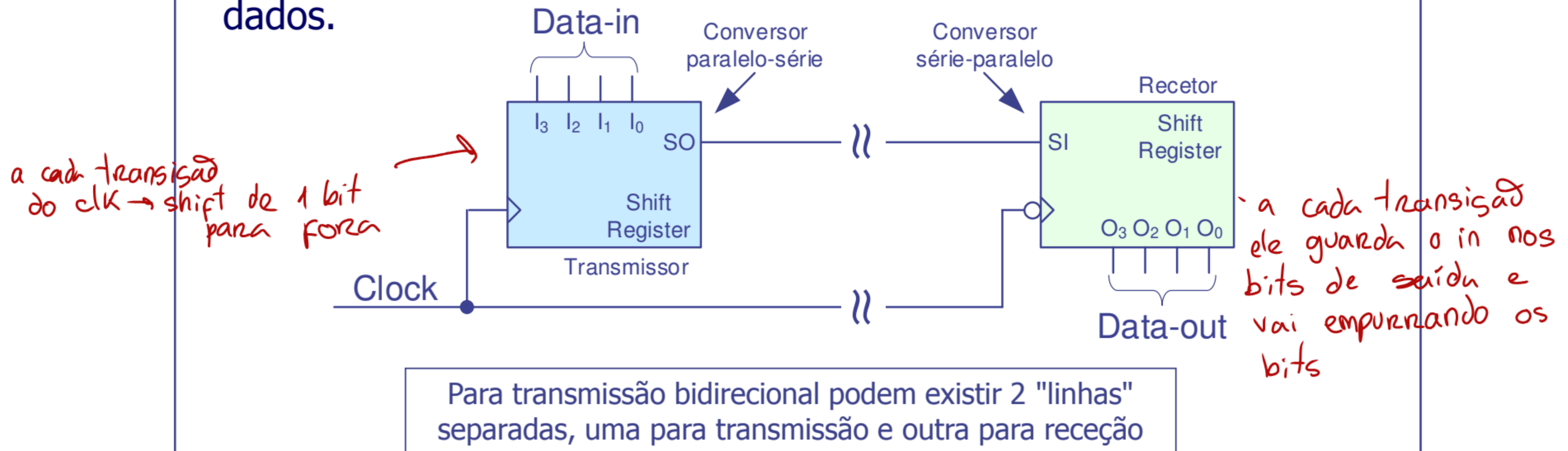


Tipos de comunicação série

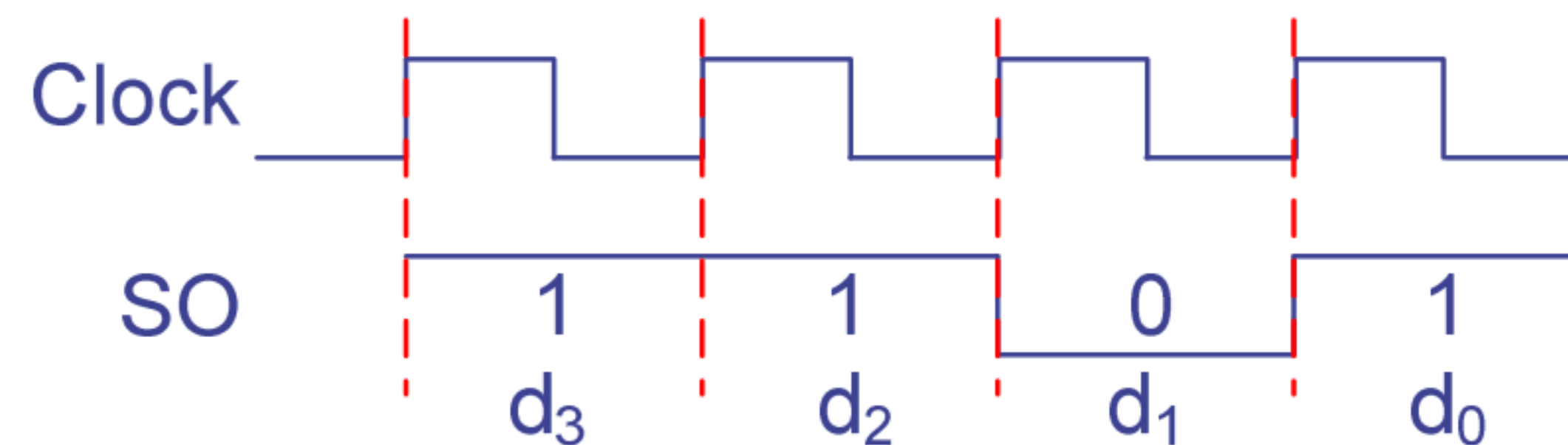
- **Simplex:** *num unico sentido*
 - Comunicação apenas num sentido (Transmissor -> Recetor); usada, por exemplo, em telemetria, para leitura remota de sensores
ex: leitura remota de sensores
- **Full-duplex:** *nos dois sentidos ao mesmo tempo (tem de ter 2 canais)*
 - Comunicação bidirecional simultânea (Transmissor -> Recetor e Recetor -> Transmissor)
 - São usados dois canais de comunicação
- **Half-duplex:** *walkie talkie* *se um canal fala o outro ouve*
 - Comunicação bidirecional (nos dois sentidos), mas não simultaneamente
 - É usado um único canal de comunicação
 - Requer comutação entre modo de transmissão e modo de receção

Exemplo de comunicação série

- Diz-se que se está na presença de um barramento ou interface série sempre que exista uma só "linha" (suporte) para transferência de dados.



- Exemplo
(Data-in = 1101)

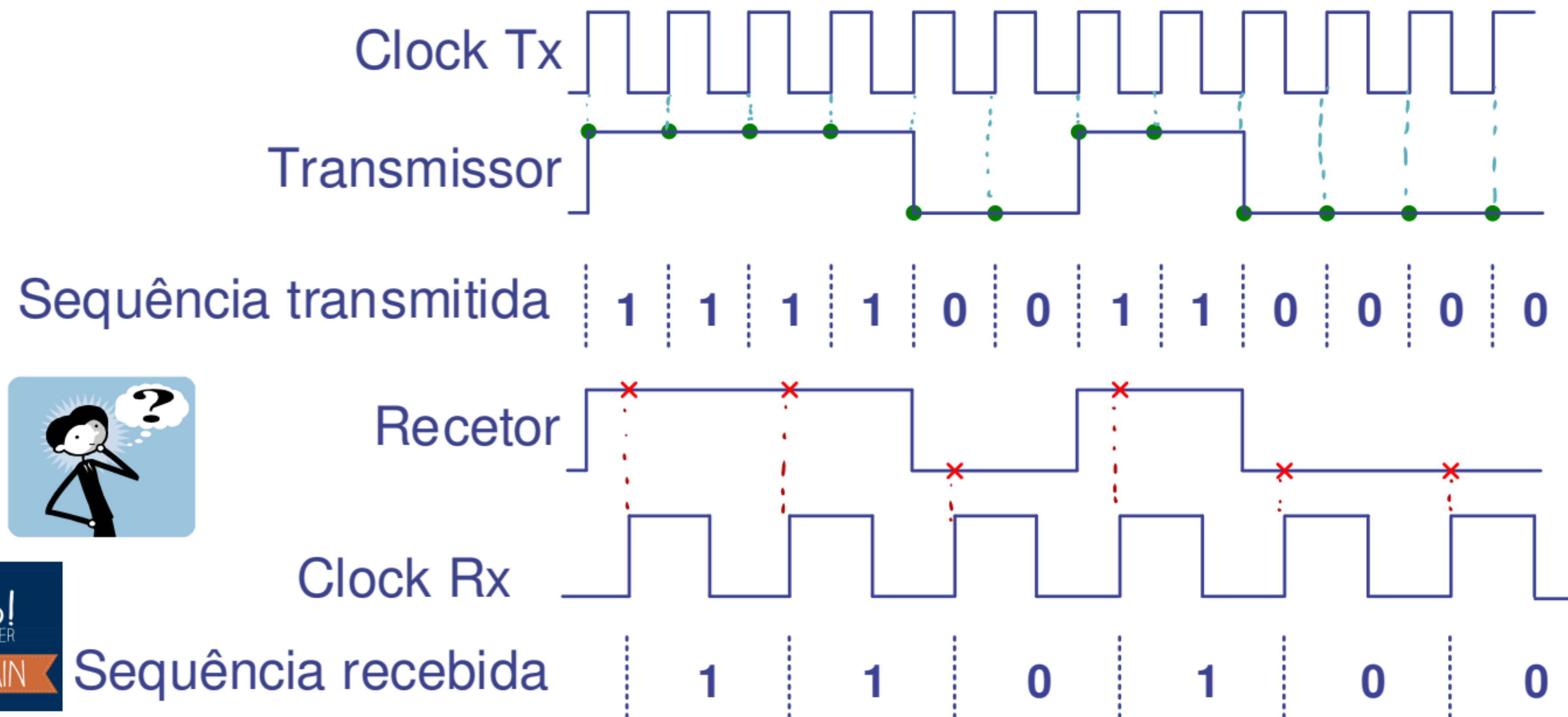


Barramentos série *termos a lembrar*

- **Transmissor** – dispositivo que envia dados para o barramento
- **Recetor** – dispositivo que recebe dados do barramento
- **Master** – dispositivo que inicia e controla a transferência de dados, determinando o início e o fim da comunicação
- **Slave** – o dispositivo endereçado pelo *master*
↳ responde a pedidos do master
- Barramento **Multi-master** – barramento em que múltiplos dispositivos podem atuar como *masters*, tentando simultaneamente controlar o barramento sem corromper a comunicação em curso
 - **Arbitragem** – mecanismo que assegura que, se mais do que um *master* tentar controlar o barramento simultaneamente, apenas um poderá continuar, sem que haja perturbação da comunicação iniciada pelo *master* vencedor
 - **Sincronização** – procedimento para sincronizar os sinais de relógio dos dispositivos envolvidos na comunicação, garantindo um funcionamento coordenado do barramento
↳ entre transmissor e recetor

Sincronização do relógio

- O sincronismo é obtido através da utilização do mesmo relógio no transmissor e no recetor, ou de relógios independentes que terão que estar sincronizados durante a transmissão



- Caso sejam distintos, os relógios do Transmissor e do Recetor têm de estar sincronizados para que a amostragem do sinal seja realizada nos instantes adequados (instantes de amostragem)

Sincronização do relógio

- **Transmissão Síncrona**

- O sinal de relógio é transmitido de forma explícita através de um sinal adicional, ou codificado nos dados *implicita*
- Os relógios do transmissor e do recetor têm de se manter sincronizados
- Quando o relógio não é explicitamente transmitido, o relógio do recetor é recuperado a partir das transições de nível lógico na linha de dados

- **Transmissão Assíncrona** *- transmissor e recetor têm clocks diferentes
só funciona quando os dois tenham frequências próximas*
 - Não é usado relógio na transmissão, nem há recuperação do relógio na receção
 - É necessário acrescentar bits para sinalizar o princípio e o fim da transmissão (e.g. "start bit", "stop bit"), que permitam ao recetor proceder à amostragem do sinal recebido, com o menor erro temporal possível

Técnicas de sincronização do relógio

- Transmissão síncrona

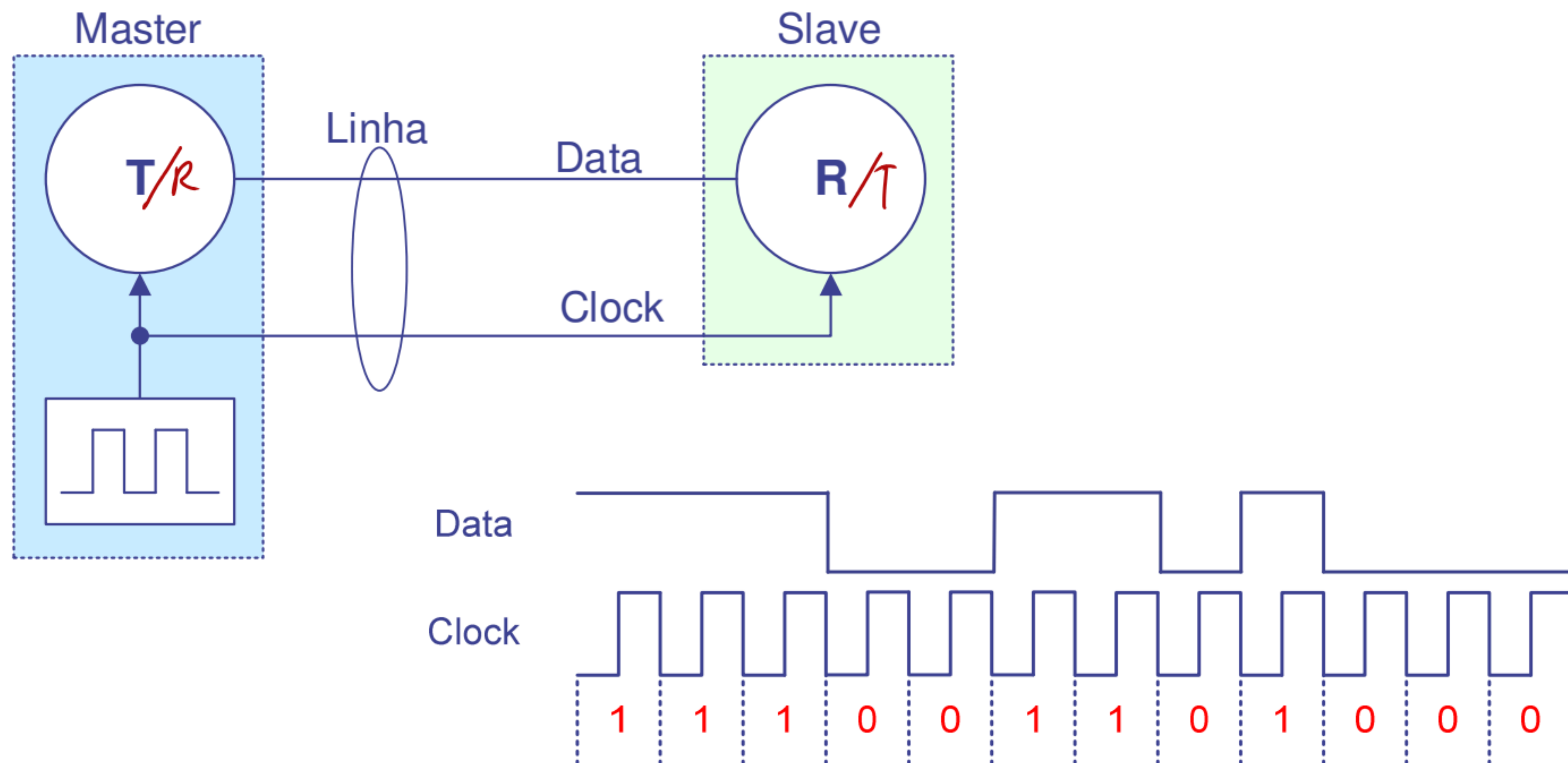
- **Relógio explícito (gerado pelo dispositivo "master")** - ligação física do relógio entre transmissor e receptor
 - Exemplo: SPI
 - **Relógio explícito mutuamente-sincronizado**
 - Exemplo: I2C
 - **Relógio codificado ("self-clocking")**
 - Exemplo: USB, Ethernet
- nos dados transmitidos o relógio vai codificado
receptor "descodifica" o relógio

- Transmissão assíncrona

- **Relógio implícito**
 - Exemplo: RS-232, CAN
↳ automóveis p.ex

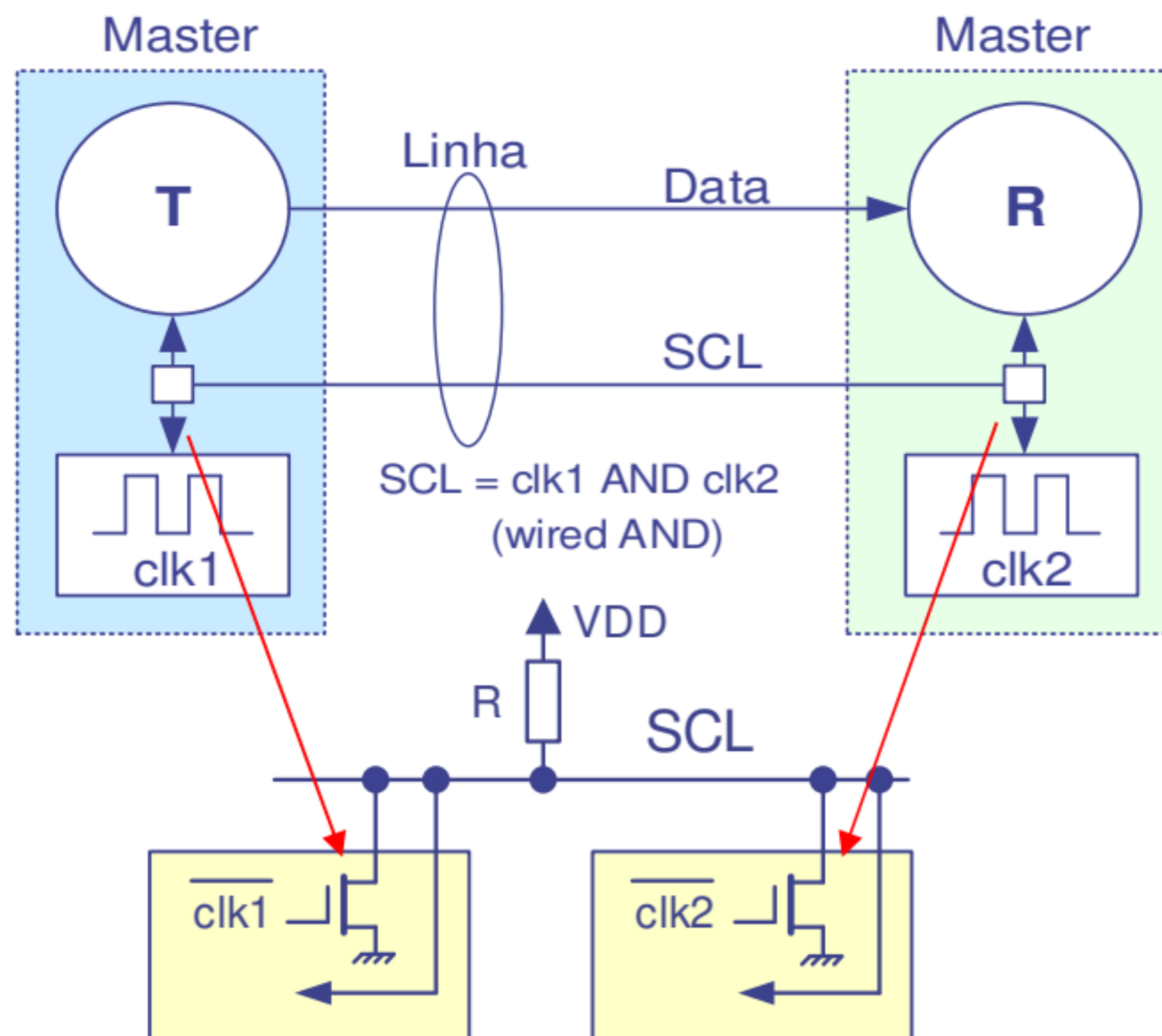
Relógio explícito gerado pelo "master"

- Linhas de relógio e dados separadas
- O sinal de relógio determina os instantes de envio/armazenamento dos dados



Relógio explícito mutuamente-sincronizado

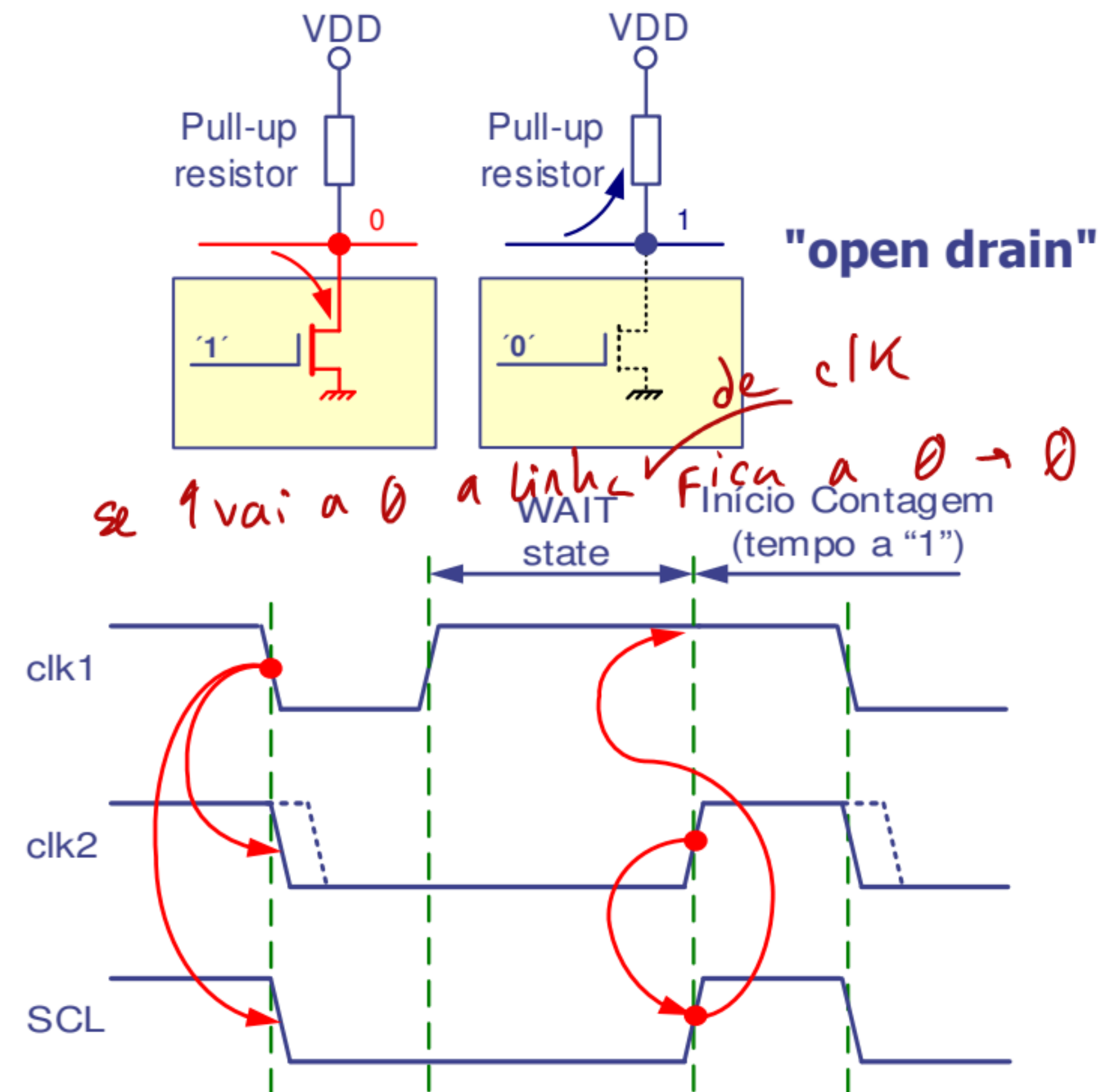
- Dois ou mais Master sincronizam-se mutuamente (vários Masters partilham o barramento)



- Em comunicação master-slave:

- O Master gera o relógio
- O Slave pode manter a linha SCL a 0 para "esticar" o ciclo de relógio: **"clock stretching"**; o Slave usa o relógio gerado pelo Master, mas pode atrasá-lo

tempo a 0 é imposto pelo master com maior tempo a 0



se 1 vai a 0 a linha fica a 0 → 0 é dominante

tempo a 1 é pelo master com menor tempo a 0 (?)

Relógio explícito mutuamente-sincronizado

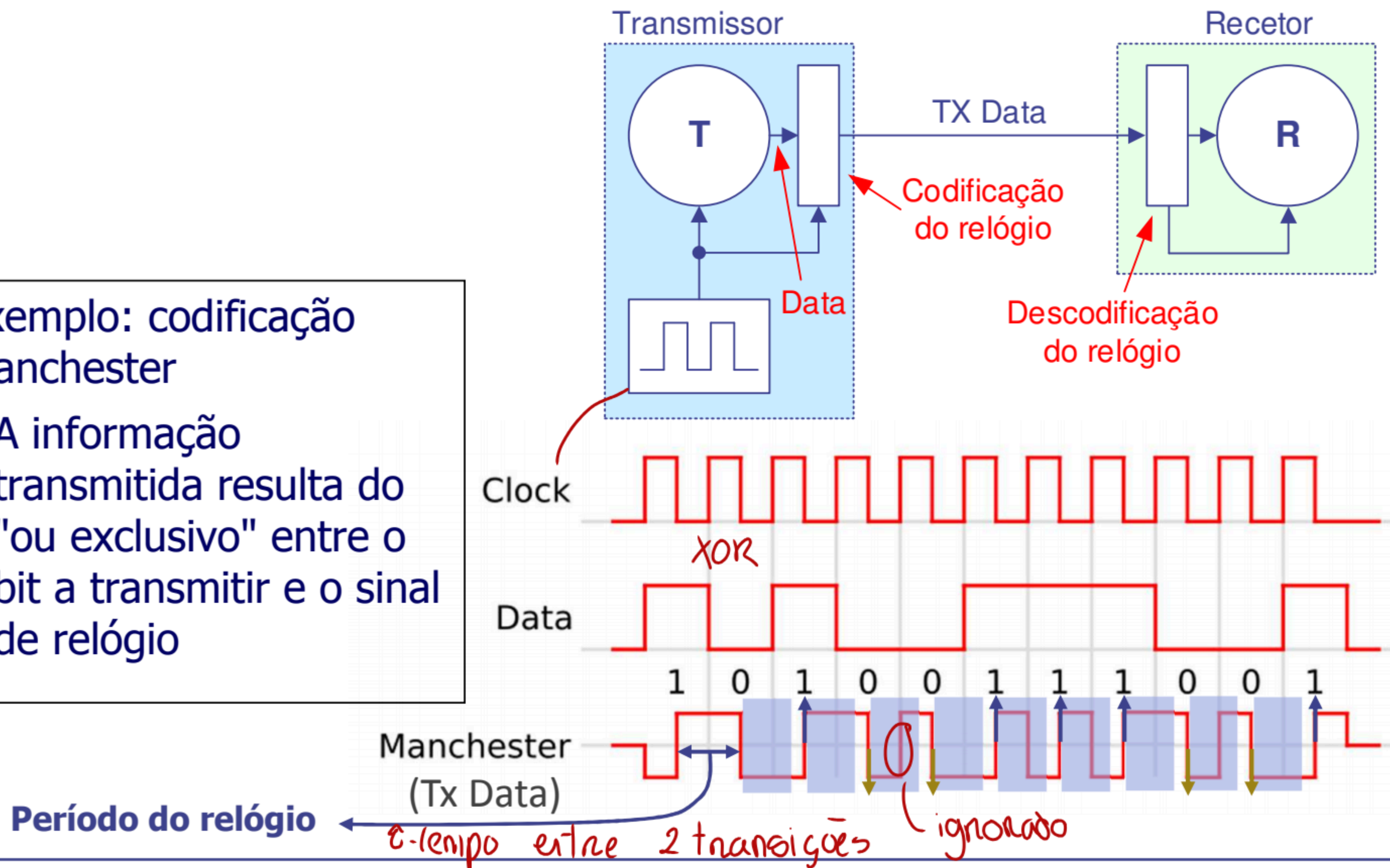
- Quando a linha SCL passa de 1 -> 0 todos os masters colocam os seus relógios a 0
- Cada master mantém o relógio a 0 até terminar o seu tempo correspondente
- Assim que um master termina o tempo a 0, liberta a linha SCL (permitindo que esta volte a 1)
- Se a linha SCL ainda se mantiver a 0, o master entra num estado de "wait", aguardando que a linha passe a 1.
- Logo que a linha SCL passe a 1, o master inicia a contagem do tempo a 1
- O primeiro master a terminar o tempo a 1 força novamente a linha SCL a 0
- O sinal SCL fica sincronizado com um t_{low} determinado pelo *master* com maior t_{low} e um t_{high} imposto pelo master com menor t_{high}
- Assim, todos os masters permanecem sincronizados, mesmo com diferentes frequências internas de relógio

Relógio codificado *só funciona bem em frequências baixas*

- Relógio enviado, em forma codificada, conjuntamente com os dados

Exemplo: codificação Manchester

- A informação transmitida resulta do "ou exclusivo" entre o bit a transmitir e o sinal de relógio

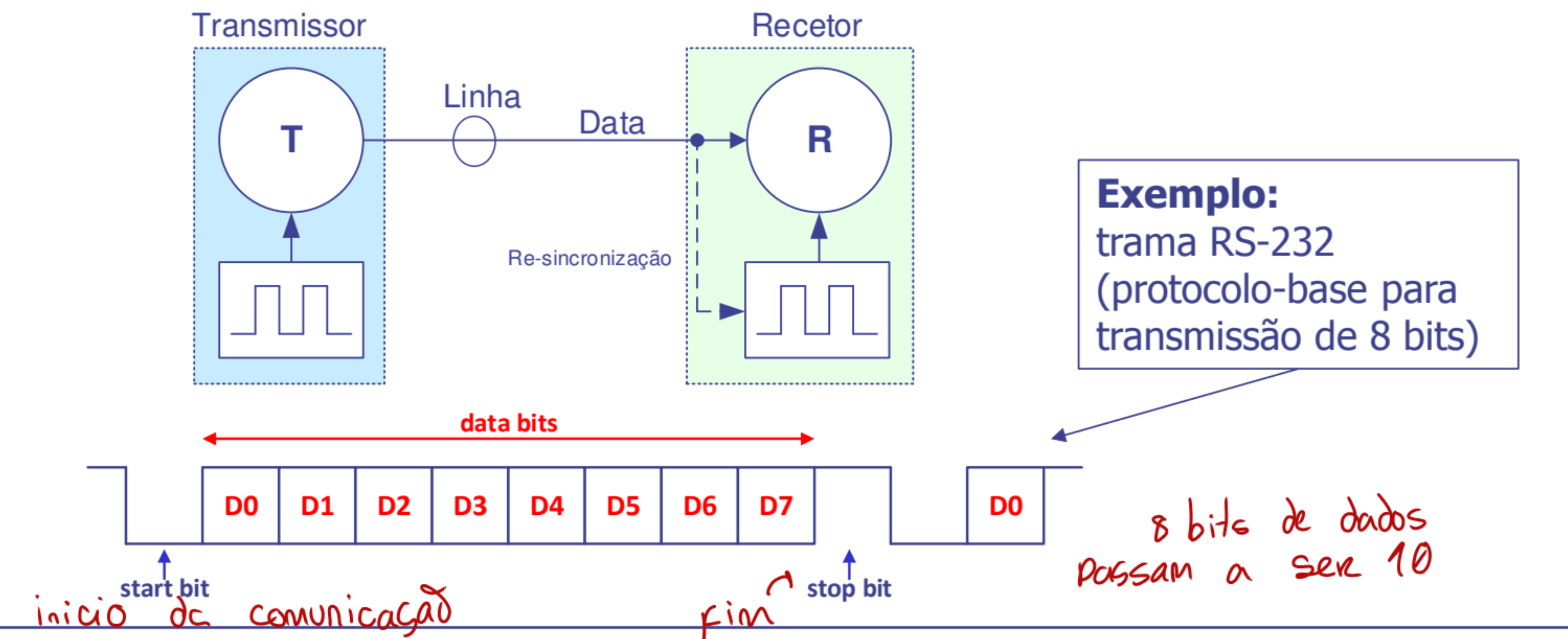


Sincronização de relógio (transmissão assíncrona)

só funciona com sistemas com freq. próximas

- **Relógio implícito**

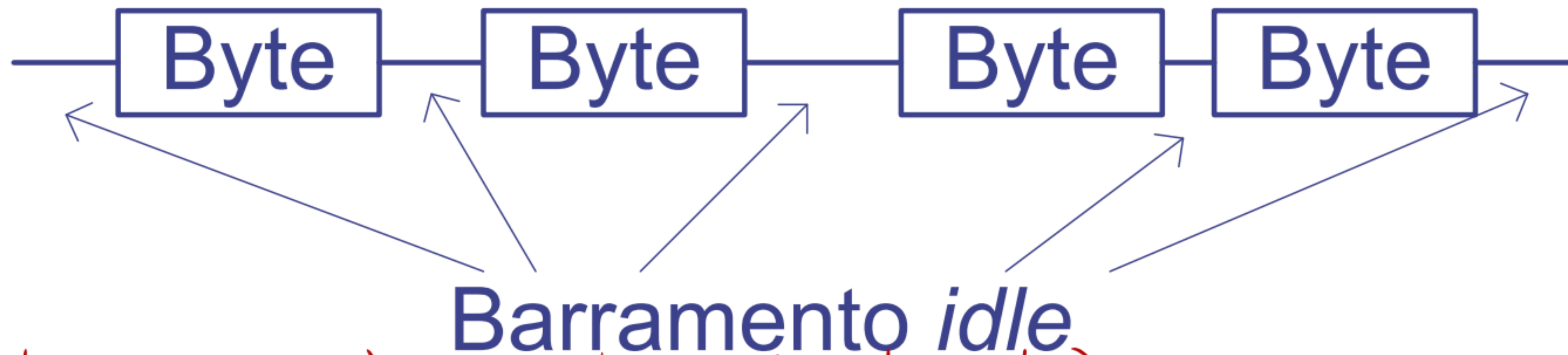
- Os relógios são locais (i.e. não há comunicação do relógio)
- O relógio do recetor é sincronizado ocasionalmente com o do transmissor por meio da receção de símbolos específicos
- Entre instantes de sincronização o desvio dos relógios depende da estabilidade/precisão dos relógios do transmissor e do recetor



Transmissão de dados – transmissão orientada ao Byte

mínimo tamanho a transmitir – 1 byte

- O envio de um byte é a operação atômica (indivisível) do barramento – o byte é transmitido na totalidade sem interrupção
- Cada byte é tratado como independente dos restantes, sendo a sua interpretação definida pelo protocolo de comunicação
- Não há restrições temporais para a transmissão de bytes consecutivos



transmissão pode ser interrompida (entre bytes)

- Alguns bytes podem estar reservados para estruturar a informação (por ex. número de bytes da sequência, soma de controlo, ...)
- Exemplo de transmissão orientada ao byte: protocolo RS-232

pode haver:

- campo a dizer quando começa a transmissão
- campo a dizer quantos bytes são mandados

Transmissão de dados – transmissão orientada ao bit

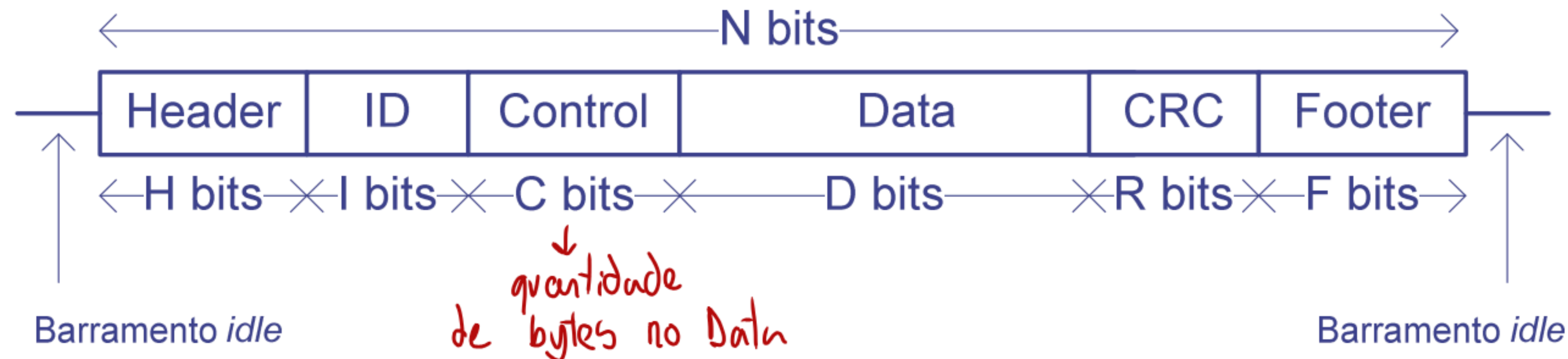
de qualquer comprimento

- A informação é organizada em tramas (sequência de bits intercalada entre duas situações de meio livre)
- As tramas são constituídas por um símbolo de sincronização (delimitador, constituído por 1 ou mais bits) seguido por uma sequência de bits de comprimento arbitrário
- As tramas podem conter campos com diferentes funções:
 - Sincronização: sinalização de início e de fim da trama
 - Arbitragem de acesso ao meio (em barramentos "multi-master")
 - Identificação. Diversas formas possíveis:
 - Quem produz
 - Qual o destino
 - Identificação da informação que circula na trama
 - ...
 - Quantidade de informação transmitida
 - Dados
 - Detecção de erros de transmissão

-transmissão n pode ser interrompida!

Transmissão de dados – transmissão orientada ao bit

- Exemplo de estrutura de uma trama



- Exemplo de transmissão orientada ao bit: barramento CAN ("Controller Area Network")
- "Header" e "footer": delimitadores de início e fim de trama
- Data: campo de dados
- CRC ("cyclic redundancy check"): código usado para detetar, no recetor, erros na comunicação
 - Uma forma simples de CRC consiste em somar todos os bytes transmitidos (soma truncada com R bits) – *checksum*
 - O recetor calcula o seu próprio *checksum* e compara com o *checksum* recebido → se der erro o recetor sinaliza e o transmissor reenvia

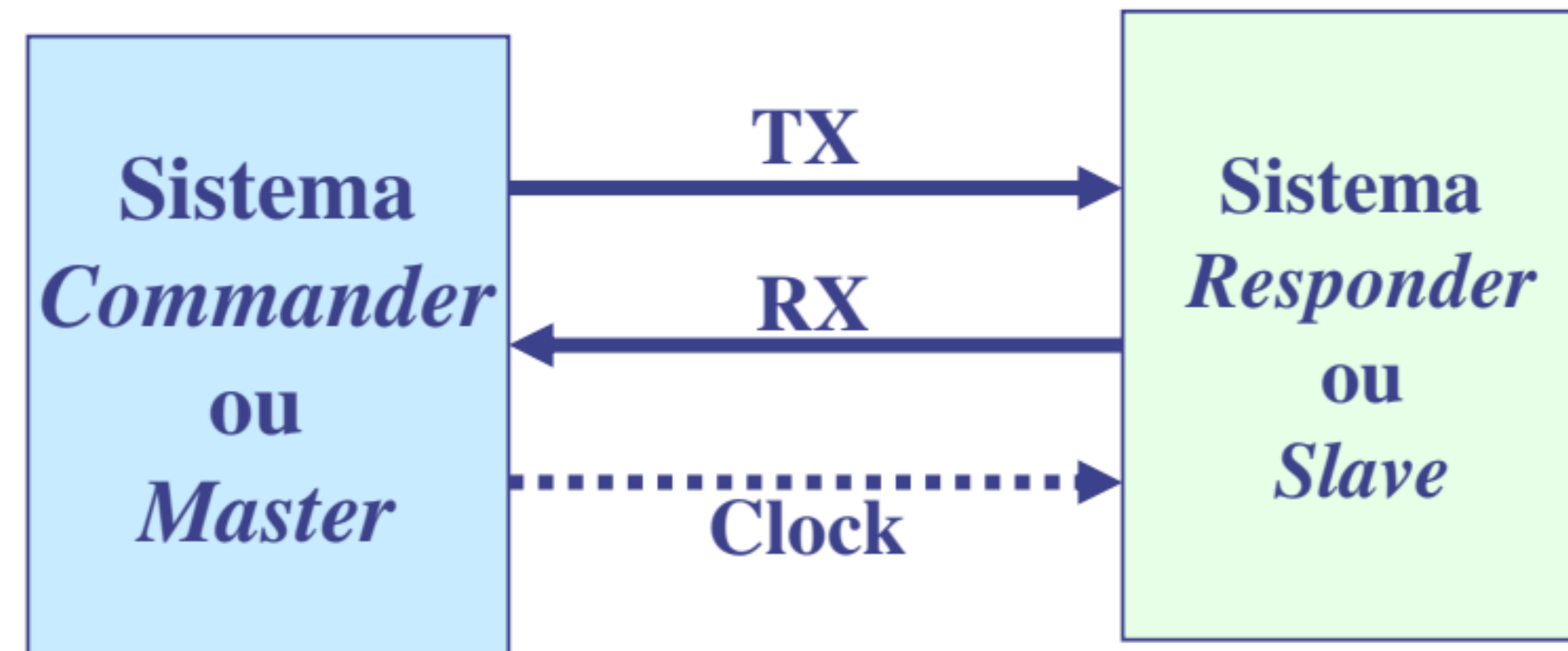
Topologias

um canal de comunicação (um sentido por vez)
↑

- Comunicação ponto a ponto ("half-duplex" ou "full-duplex"):

↳ sempre dois sistemas

↳ dois canais de comunicação

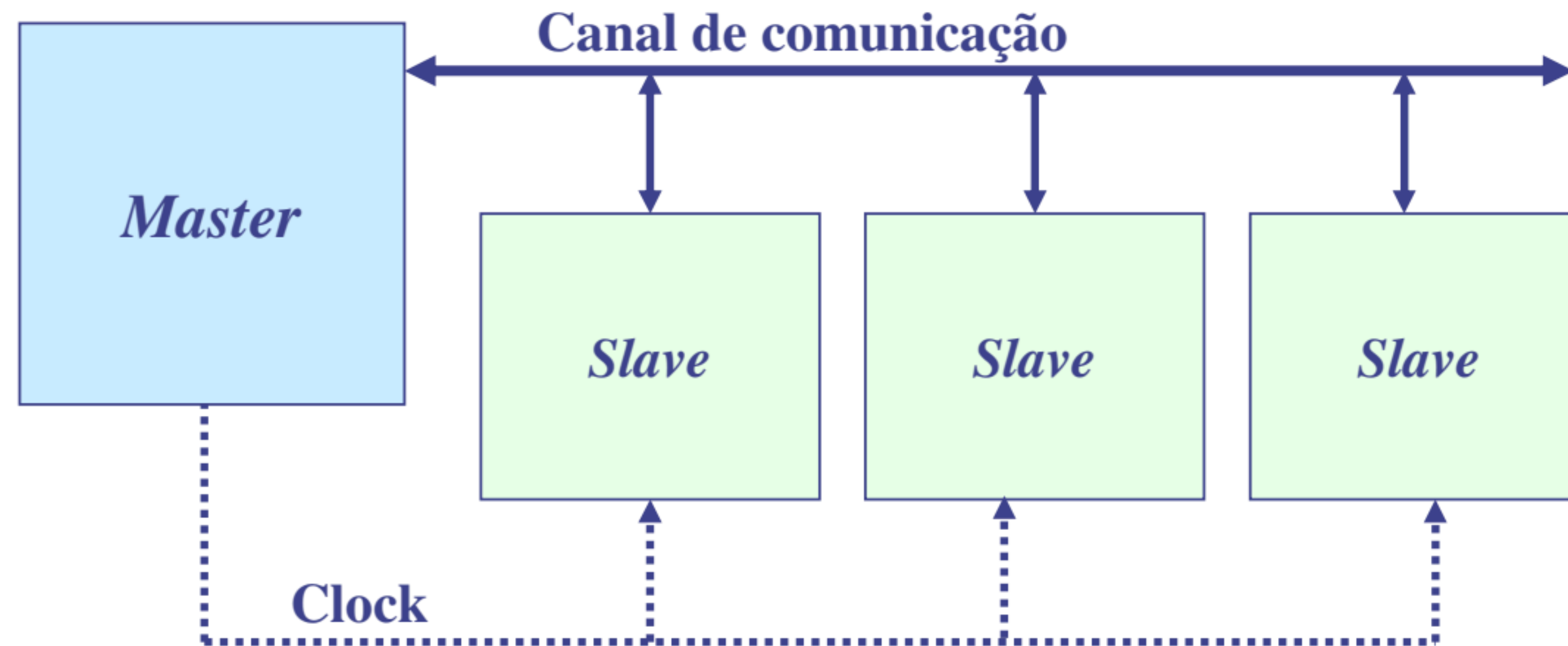


- Ligação direta entre dois dispositivos, permitindo a troca de dados em ambos os sentidos:
 - No modo half-duplex, a comunicação ocorre alternadamente (num sentido de cada vez)
 - No modo full-duplex, a transmissão e a recepção decorrem em simultâneo

Topologias

↑ 1 canal de comunicação

- Comunicação multiponto ("half-duplex"):



- Vários dispositivos partilham a mesma linha de comunicação
- Apenas um pode transmitir de cada vez; os restantes permanecem em escuta
- A comunicação ocorre num único sentido de cada vez
- É necessário um mecanismo de arbitragem ou controlo para evitar colisões entre transmissões

• Todos os slaves recebem a info? depende do sistema → só quem precisa ouve
→ o master diz para quem vai

Elementos de uma ligação série

- Exemplo de uma ligação série entre um sistema embutido ("embedded" ou dedicado) e um computador de uso geral (PC)

